Dr. Karl Fürlinger
Lehrstuhl für Kommunikationssysteme und Systemprogrammierung

The C++ Memory Model and Atomic Operations

Part 2



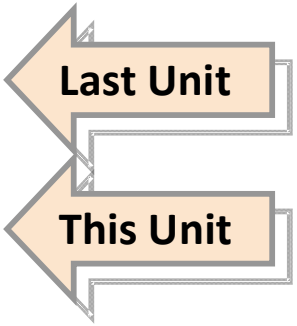
Recap: Memory Locations and Data Races

- Objects in memory are composed of one or more **memory locations**
  - Fundamental types occupy exactly 1 memory location
  - Adjacent bitfields share the same memory location

```
struct S {
    char a;      // mem. loc. #1
    int b : 5;   // mem. loc. #2
    int c : 11, // mem. loc. #2 (cont.)
          : 0,
        d : 8;  // mem. loc. #3
    struct SS {
        int ee : 8; // mem. loc. #4
    } e;
} obj;
```

Example from www.cppreference.com

- Data Races
  - Two memory access operations to the **same memory location** by **two different threads**, where at least one is a write operation, are said to be **conflicting**
  - Two conflicting accesses constitute a **data race**, unless
    - **Both** accesses are performed through **atomic operations**, or
    - One conflicting access ***happens-before*** the other



Happens-Before Relationship

- In order to avoid data races, we need to establish a *happens-before* relationship between operations

- Let A and B represent operations performed by a multithreaded program. If A ***happens-before*** B, then the memory effects of A effectively become visible to the thread performing B, before B is performed.

- Discussion:
  - *Happens-before* is only concerned with the memory effects. We want to guarantee that changes performed by one thread become visible to another thread.
  - The name sounds like it refers to time -- but, strictly speaking, the notion of *happens-before* is independent of any temporal ordering of events (see discussion later)

Happens-Before Within a Single Thread

- Within a single thread, the *happens-before* order is simply established by the order of statements in the program (program order)
  - Note: the evaluation order of expressions within a statement is not always guaranteed by C++ rules
  - Again, all we care about is that the effects of an earlier operation are visible to a later operation
  - In reality operations may be reordered, as long as the *happens-before* condition is met

```
x=1;   // A
y=42;  // B
z=x+2; // C
```

```
// could actually be executed
// with reordering like this:
x=1;   // A
z=x+2; // C
y=42;  // B
```

- In this example:
  - A *happens-before* B
  - B *happens-before* C
  - A *happens-before* C

Happens-Before Between Threads

- How to establish the *happens-before* relationship between operations in different threads?
  - We need a new relationship, called ***synchronizes-with***
  - *Synchronizes-with* is **based on atomic operations**

- **Definition (simplified):** A "suitably tagged" **atomic write** operation W of a variable x ***synchronizes-with*** a "suitably tagged" **atomic read** operation of x **that reads the value stored by W**
  - "suitably tagged" refers to the memory ordering parameters that atomic operations can have (more on this later)

- We use *synchronizes-with* to extend *happens-before* to other threads:
  - If A *synchronizes-with* B, then A *happens-before* B
  - Transitivity: If A *happens-before* B and B *happens-before* C, then A *happens-before* C

Example (Flag-Based Synchronization)

- Without using atomic operations:

```
int val{0};
bool flag{false};

// Thread 0                 // Thread 1
val=42;                     while(!flag) {/*spin*/}
flag=true;                  cout << val;
```

Data Race! Undefined Behavior!

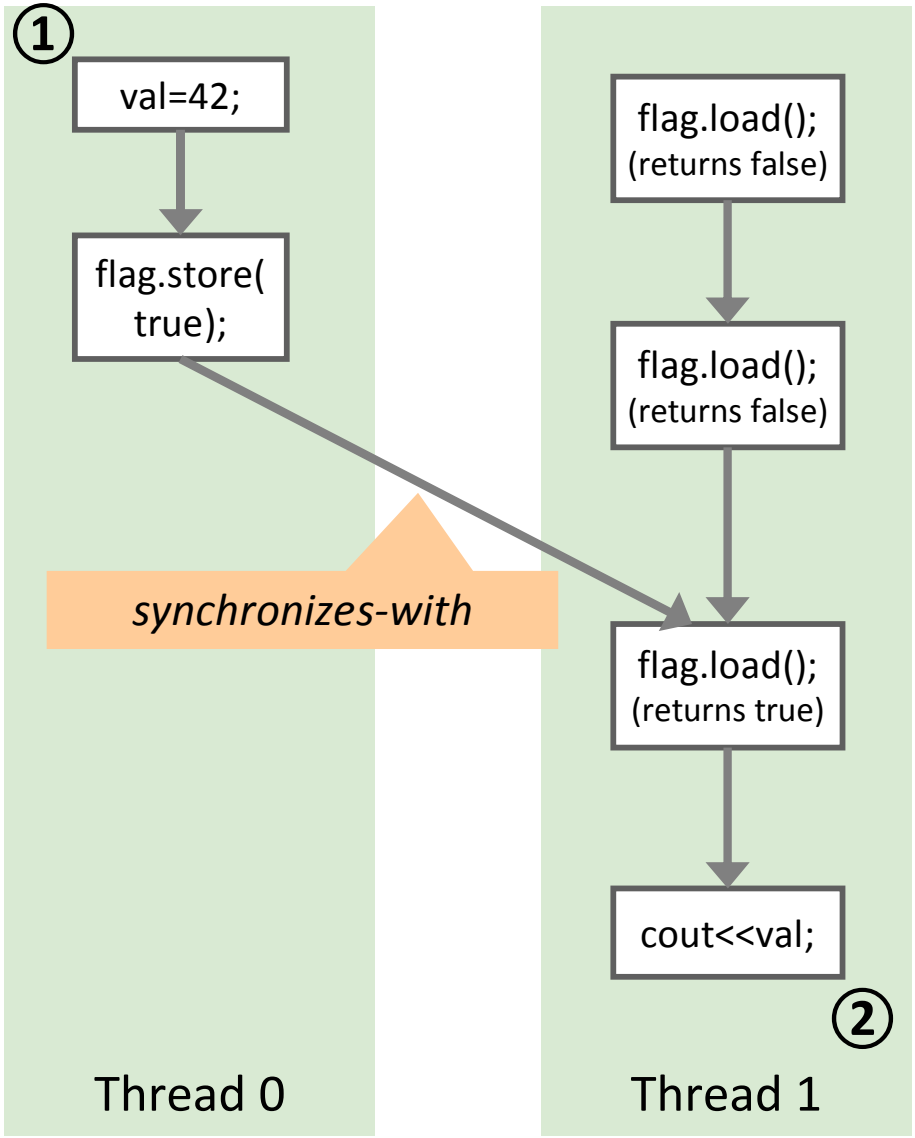
- Using atomic operations:
  - Remember that the default ordering for atomic operations is memory_order_seq_cst

```
int val{0};
std::atomic<bool> flag{false};

// Thread 0                 // Thread 1
val=42;                     while(!flag.load()) {/*spin*/}
flag.store(true);           cout << val;
```

No Data Race!

Visualization of the Previous Example



① ***happens-before*** ②,
as desired!

Inspired by Anthony Williams "C++ Concurrency in Action"

Happens-before != Happens Before

- *Happens-before* is independent of temporal ordering of operations
- *Happens-before* does not imply temporally happens before
  - E.g., compiler can perform reordering as long as happens-before is not violated

```
x=1;   // A
y=42;  // B
z=x+2; // C
```

```
// could actually be executed
// with reordering like this:
x=1;   // A
z=x+2; // C
y=42;  // B
```

- Temporally happens before does not imply *happens-before*
  - E.g., store buffer: the effects of an earlier store may not be visible to a later read

Recap: Memory Ordering Parameters

- Six different mem. ordering parameters, grouped into three models
  - Some do not apply to load or store operations

	Load Operation	Store Operation	RMW Operation	
memory_order_seq_cst	x	x	x	*sequentially consistent ordering*
memory_order_relaxed	x	x	x	*relaxed ordering*
memory_order_acquire	x		x	*acquire-release ordering*
memory_order_release		x	x	
memory_order_acq_rel			x	
memory_order_consume	x		x	

Discouraged in C++ 17, not discussed further here

Establishing the Synchronizes-With Relationship (1)

- When are atomic loads and stores “suitably tagged”?
  - Three cases: Sequentially consistent / relaxed / acquire-release ordering

- **Sequentially Consistent Ordering:**
  A sequentially consistent store ***synchronizes-with*** a sequentially consistent load of the same variable that **reads the value written by the store**
  - Plus, SC offers further guarantees: There is a **single total order** of all operations tagged memory_order_seq_cst seen by all threads

- **Relaxed Ordering:**
  Atomic operations performed with relaxed ordering **don’t participate** in the ***synchronizes-with*** relationship
  - Exception: relaxed RMW operations participate in release sequences (later)

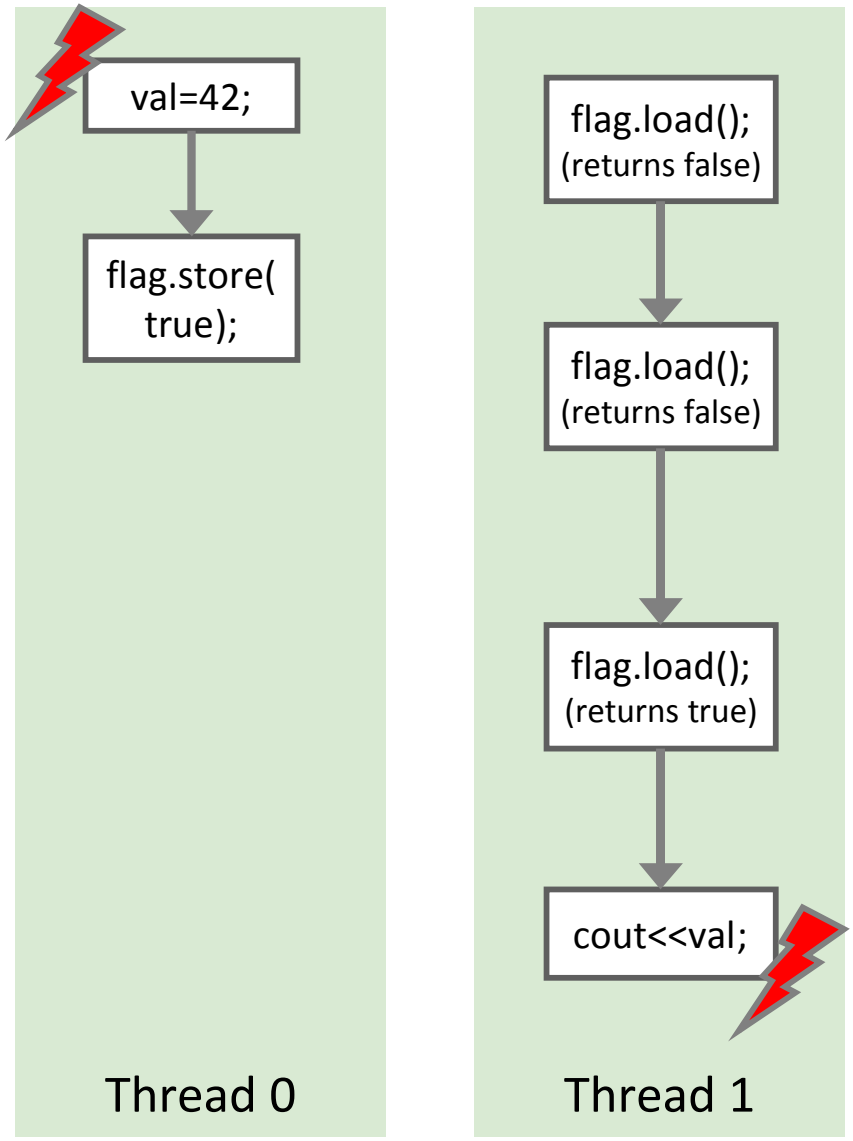
Relaxed atomics don't participate in synchronizes-with

```
int val{0};
std::atomic<bool> flag{false};

// Thread 0
val=42;
flag.store(true,
   std::memory_order_relaxed);

// Thread 1
while(!flag.load(
   std::memory_order_relaxed))
{ /* spin */ }
cout << val;
```

- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is a **data race on val**, because no *happens-before* order is established by the relaxed atomics



Establishing the Synchronizes-With Relationship (2)

- **Acquire-Release Ordering:**
  - Atomic loads are **acquire operations** (memory_order_acquire)
  - Atomic stores are **release operations** (memory_order_release)
  - Atomic RMW operations are acquire or release operations or can act as both (memory_order_acq_rel)

- A release operation (store) ***synchronizes-with*** an acquire operation (load) that **reads the value written by the store**
  - Synchronization is always **pairwise between threads**
  - Unlike SC there is **no total order of operations** that all threads agree on

- Mixing SC and acquire-release
  - SC loads behave like acquire operations
  - SC stores behave like release operations
  - SC RMW operations behave like acquire and release operations

Reordering Restrictions

- Atomic operations also limit reordering of regular (non-atomic) reads and writes:

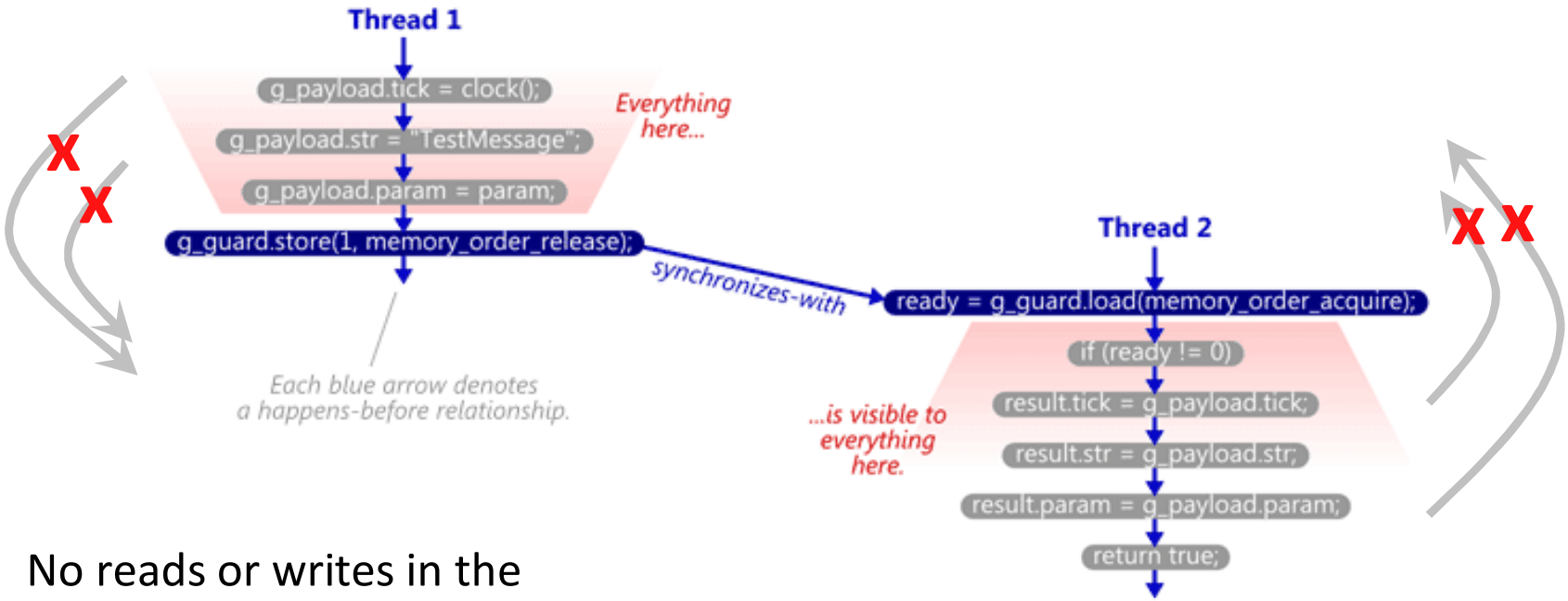


Image from https://preshing.com/

No reads or writes in the current thread can be reordered after the "release" store (RW||).

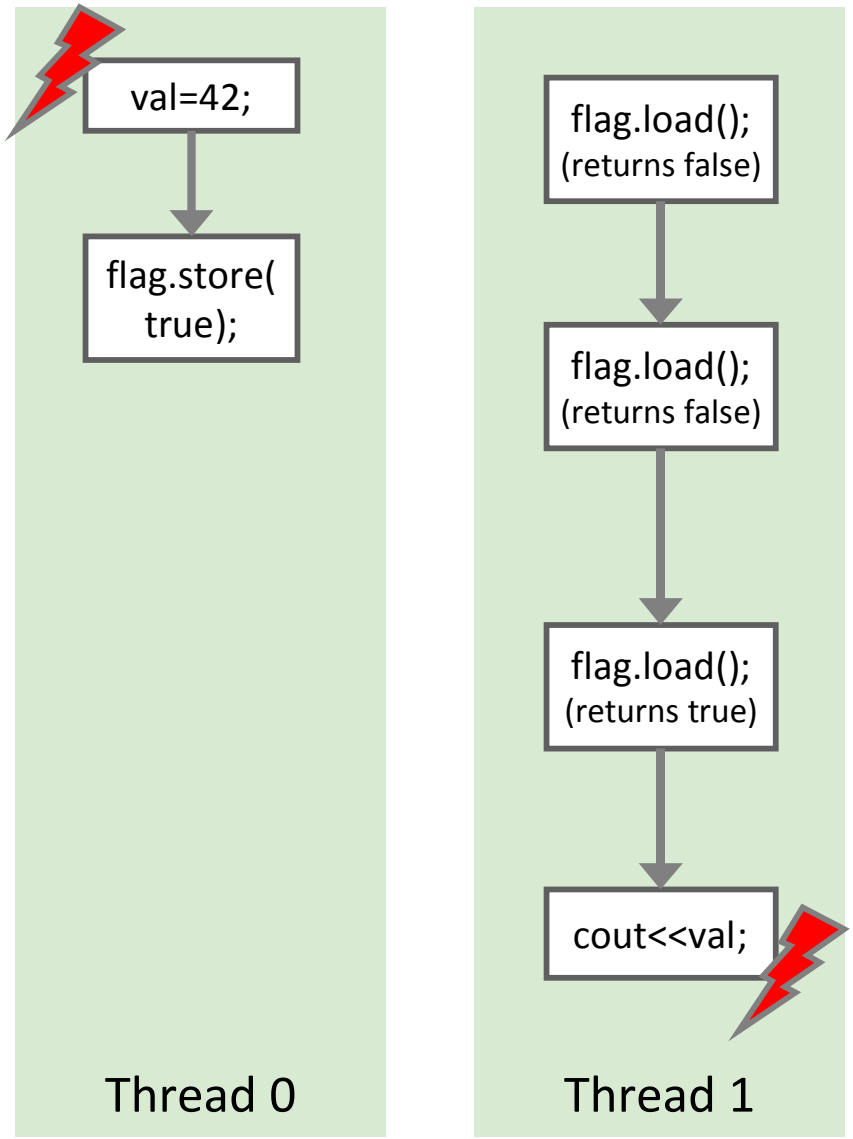
No reads or writes in the current thread can be reordered before the "acquire" load (||RW).

Examples (1)

```
int val{0};
std::atomic<bool> flag{false};

// Thread 0
val=42;
flag.store(true,
    std::memory_order_relaxed);

// Thread 1
while(!flag.load(
    std::memory_order_relaxed))
{ /* spin */ }
cout << val;
```



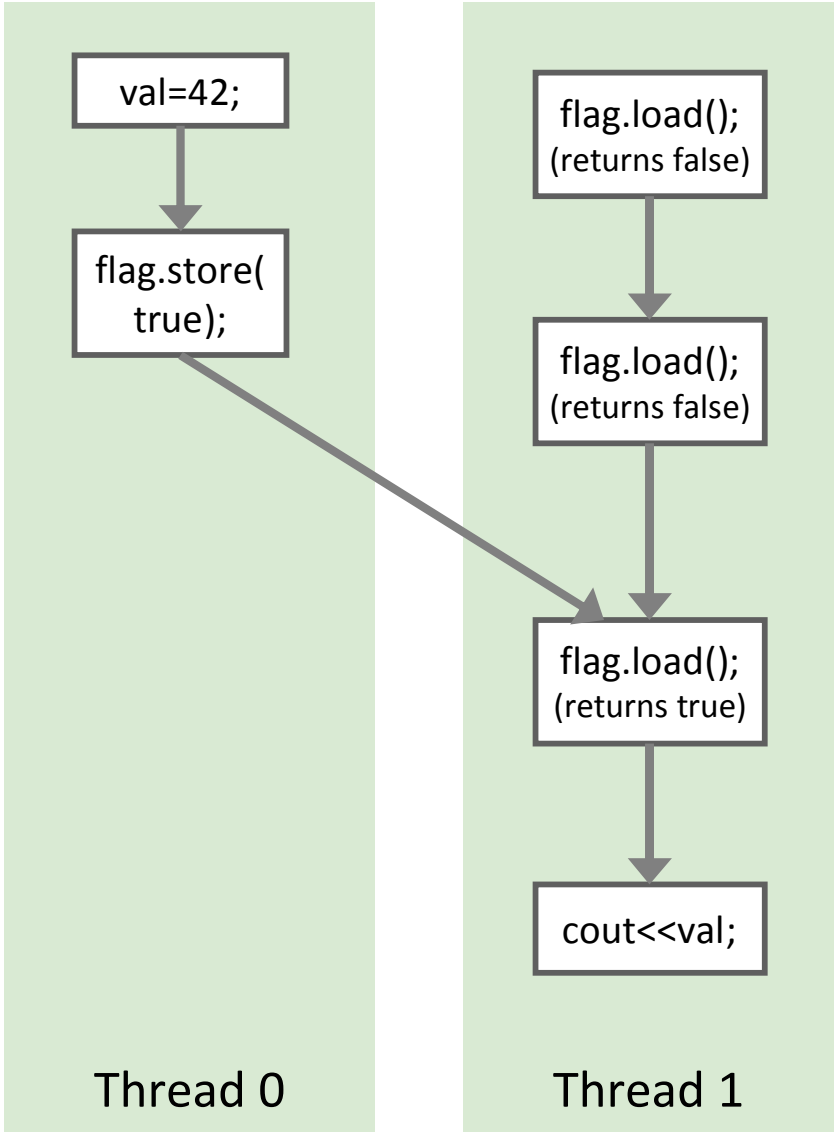
- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is a **data race on val**, because no *happens-before* order is established by the relaxed atomics

Examples (2)

```
int val=0;
std::atomic<bool> flag{false};

// Thread 0
val=42;
flag.store(true,
   std:: memory_order_seq_cst);

// Thread 1
while(!flag.load(
   std:: memory_order_seq_cst))
{ /* spin */ }
cout << val;
```



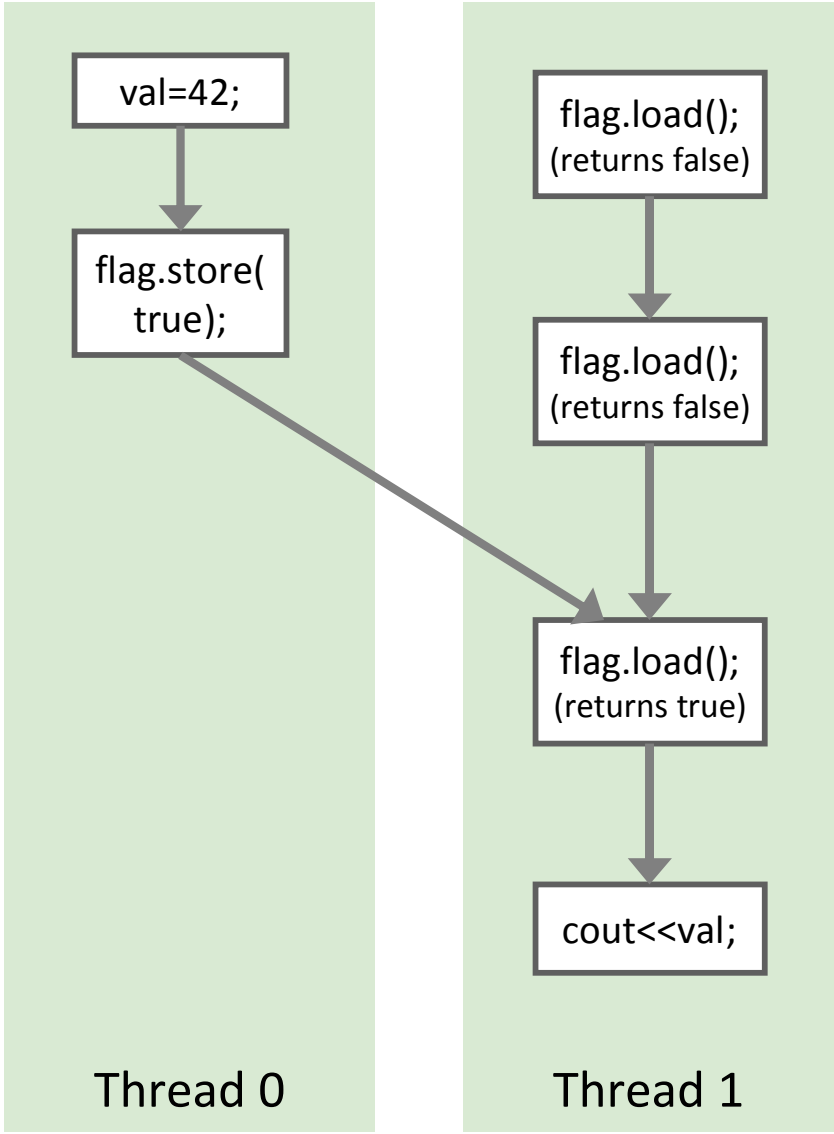
- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is **no data race on val**, because a *happens-before* order is established by the SC load-store pair

Examples (3)

```
int val=0;
std::atomic<bool> flag{false};

// Thread 0
val=42;
flag.store(true,
   std:: memory_order_release);

// Thread 1
while(!flag.load(
   std:: memory_order_acquire))
{ /* spin */ }
cout << val;
```



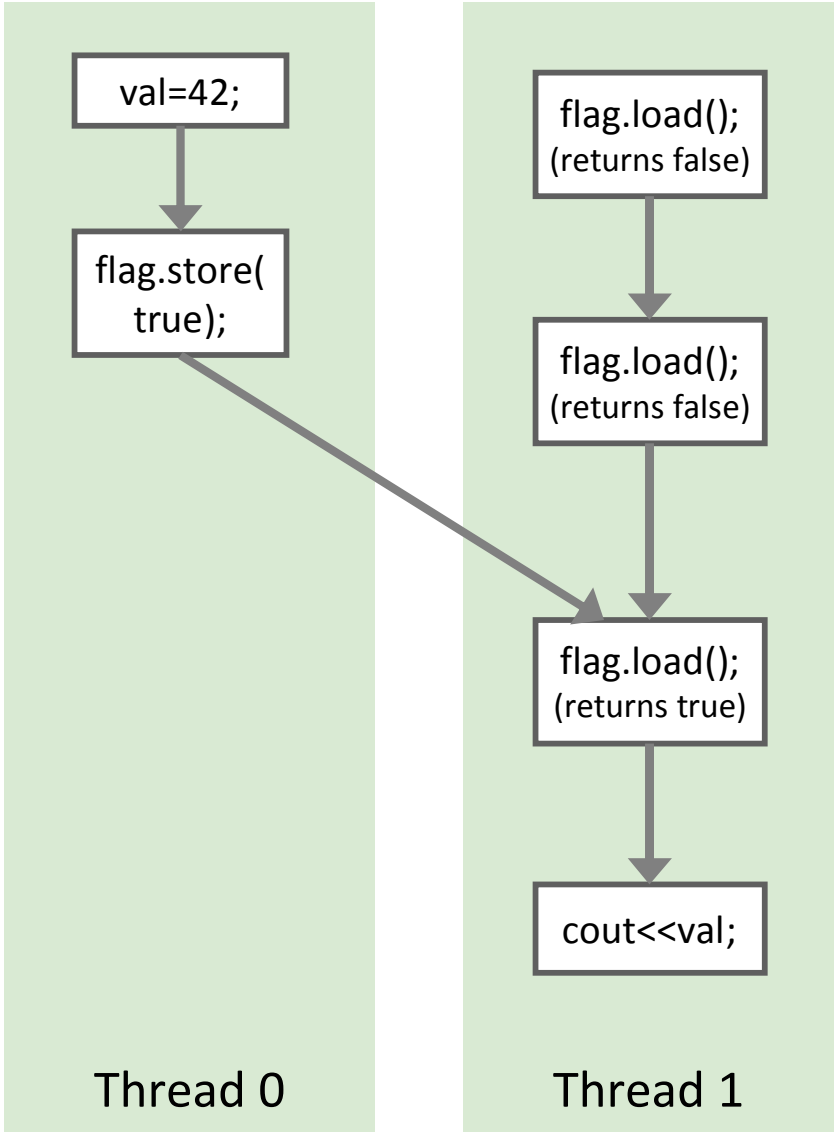
- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is **no data race on val**, because a *happens-before* order is established by the acquire-release pair

Examples (4)

```
int val=0;
std::atomic<bool> flag{false};

// Thread 0
val=42;
flag.store(true,
   std:: memory_order_release);

// Thread 1
while(!flag.load(
   std:: memory_order_seq_cst))
{ /* spin */ }
cout << val;
```



- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is **no data race on val**, because a *happens-before* order is established
  - We can mix in SC, it acts like acquire or release, as required

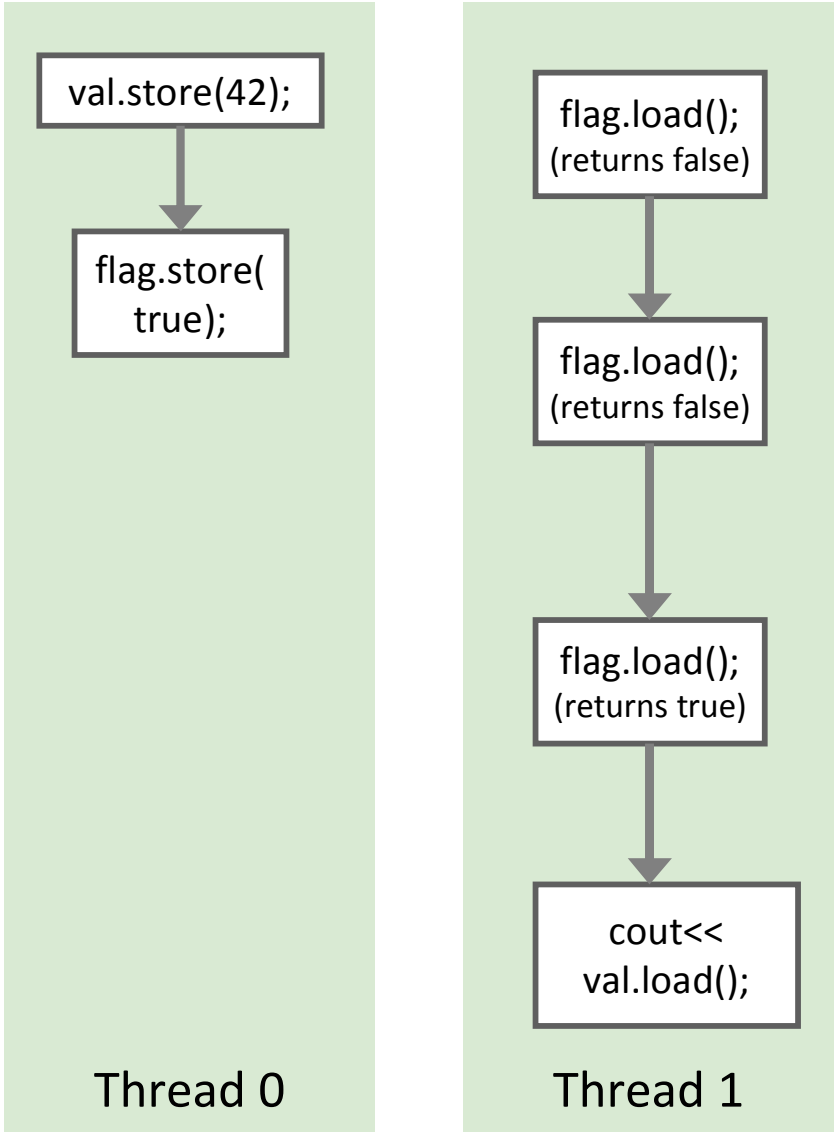
Examples (5)

```
std::atomic<int> val{0};
std::atomic<bool> flag{false};

// Thread 0
val.store(42,std::memory_order_relaxed);
flag.store(true,std::memory_order_relaxed);

// Thread 1
while(!flag.load(std::memory_order_relaxed))
{ /* spin */ }
cout << val.load(std::memory_order_relaxed);
```

- Discussion
  - There is **no data race on flag**, because atomic operations are used for access
  - There is a **no data race on val**, because atomic operations are used for access
  - But we are not guaranteed to read the "correct" value (42) in thread 1, we may read 0



Acquire-Release vs. Sequentially Consistent (1)

- Both can be used to establish the happens-before relationship between operations in different threads
  - But SC offers another guarantee: all sequentially consistent operations appear in the same total order for all threads

```
std::atomic<bool> x{false};
std::atomic<bool> y{false};

// Thread 0 (set x to true)
x.store(true, memory_order_seq_cst);

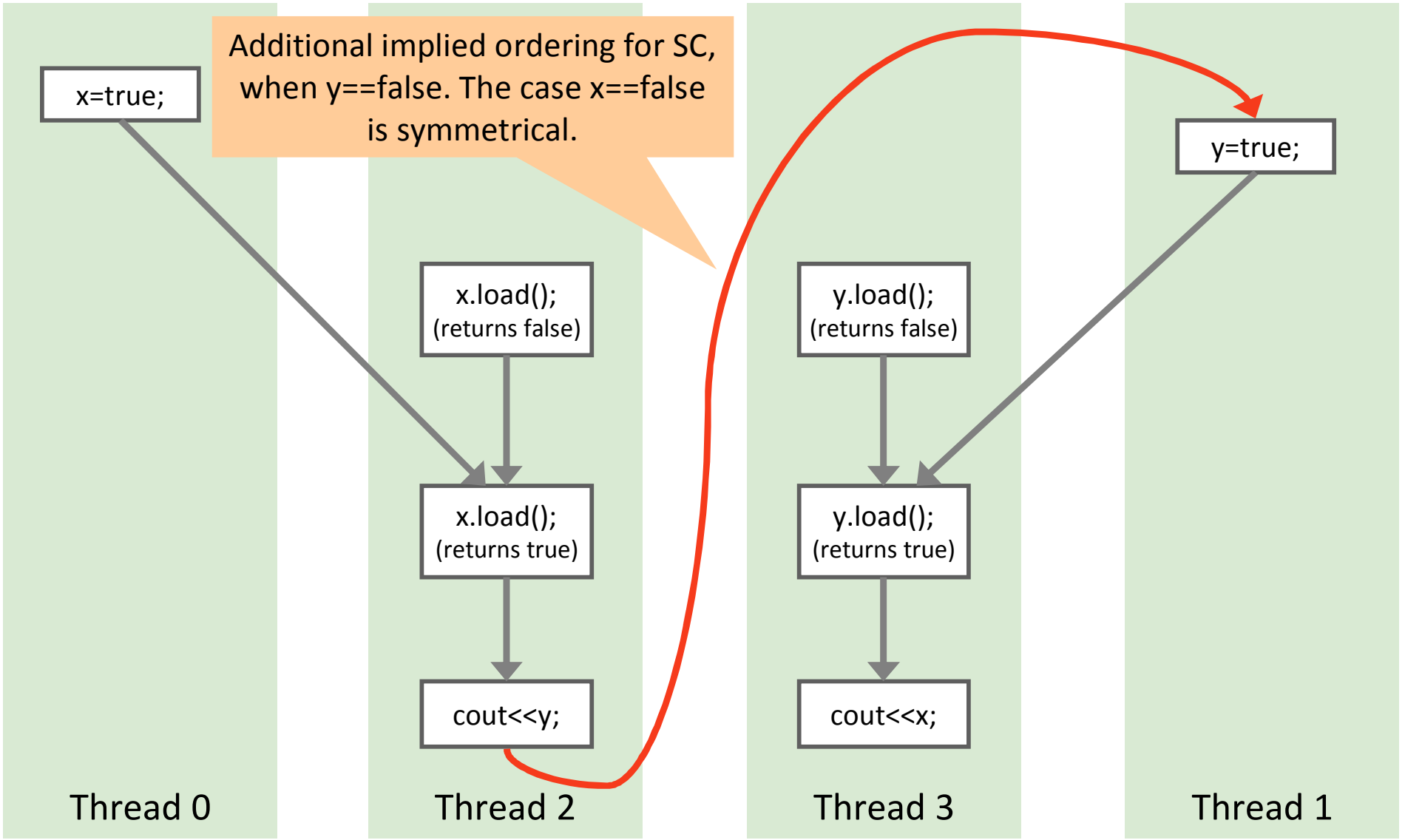
// Thread 1 (set y to true)
y.store(true, memory_order_seq_cst);

// Thread 2 (read x, then y)
while(!x.load(memory_order_seq_cst)){/*spin*/};
cout << "y==" << y.load(memory_order_seq_cst);

// Thread 3 (read y, then x)
while(!y.load(memory_order_seq_cst)){/*spin*/};
cout << "x==" << x.load(memory_order_seq_cst);
```

- Q: is x==0, y==0 a possible output?
- With SC, no!
- With Acq-Rel, yes!

Acquire-Release vs. Sequentially Consistent (2)



The *synchronizes-with* edge for x alone doesn't tell us anything about y.

The *synchronizes-with* edge for y alone doesn't tell us anything about x.

"… reads the value stored…" is important too

- Remember the definition from earlier**:**
  A "suitably tagged" **atomic write** operation W of a variable x ***synchronizes-with*** a "suitably tagged" **atomic read** operation of x that **reads the value stored** by W
- What if two threads (0, 1) store the same value and thread 2 reads that value?

```
std::atomic<bool> flag{false};

// Thread 0
...
flag.store(true,
   std:: memory_order_release);

// Thread 1
...
flag.store(true,
   std:: memory_order_release);

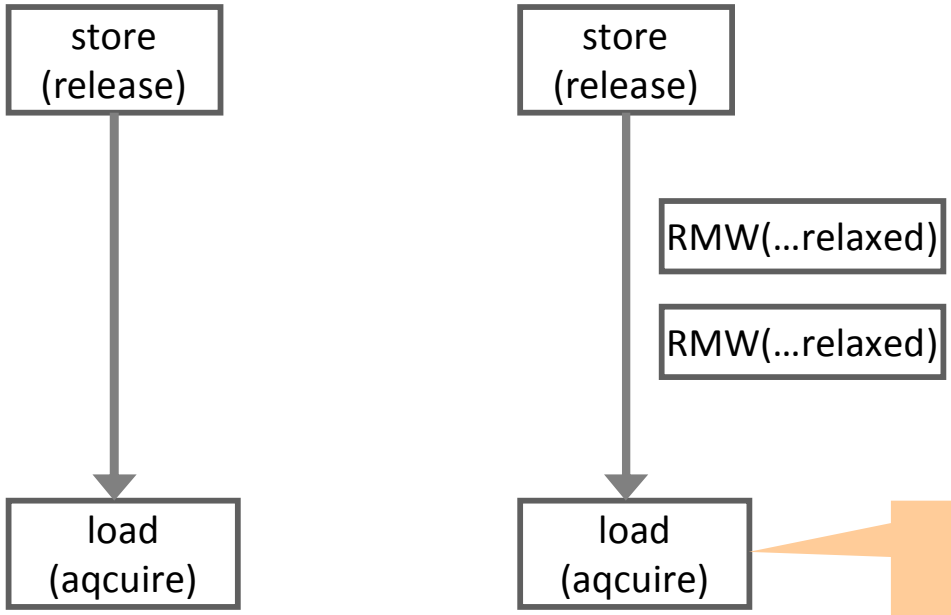
// Thread 2
while(!flag.load(
   std:: memory_order_acquire))
{ /* spin */ }
...
```

- This establishes a synchronizes-with relationship only between one pair of threads, not both! Either between threads 0 and 2 or between threads 1 and 2

Release Sequences (Overview)

- Remember our previous simplified definition for ***synchronizes-with***: "A "suitably tagged" **atomic write** operation W of a variable x ***synchronizes-with*** a "suitably tagged" **atomic read** operation of x that reads the value stored by W"

- This can be extended to cover RMW operations that happen in between the atomic write and the atomic read:



The RMW operations can have any memory ordering parameter, even memory_order_relaxed

Fences (Overview)

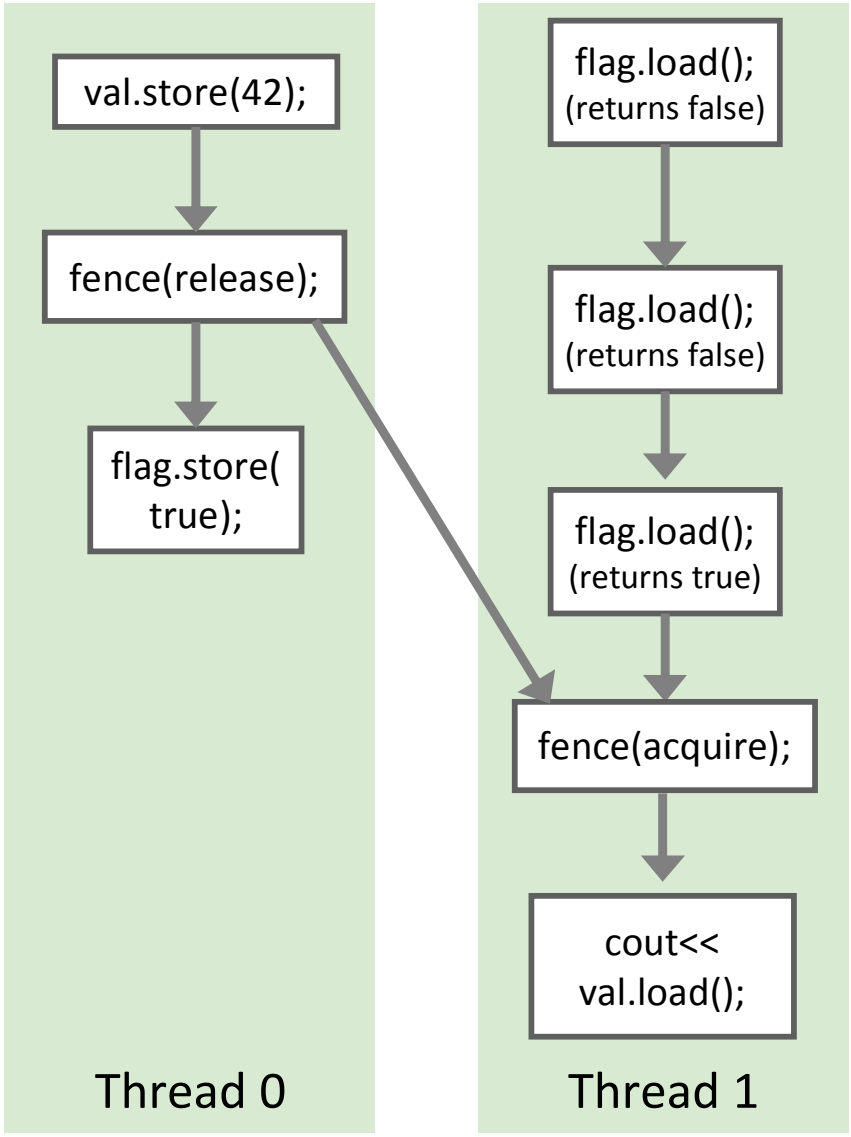
- C++ also offers a set of stand-alone memory barriers (fences)
  - std::atomic_thread_fence(std::memory_order order)
- Fences can also participate in the *synchronizes-with* relationship
  - They are typically used together with operations that use memory_order_relaxed ordering constraints
  - Acquire/release and SC load/stores can be mixed with fences
- Note: the “read sees the value of store” part of the definition of synchronizes-with is fulfilled by stores after the release fence and reads before the acquire fence (**flag** in the case of the next slide).
- Fences come with stronger reordering restrictions than acquire/release loads/stores
  - Release fence corresponds to a **RW||W** restriction
  - Acquire fence corresponds to a **R||RW** restriction

Fence (Example)

```
std::atomic<int> val{0};
std::atomic<bool> flag{false};

// Thread 0
val.store(42,std::memory_order_relaxed);
atomic_thread_fence(
        std::memory_order_release);
flag.store(true, std::memory_order_relaxed);

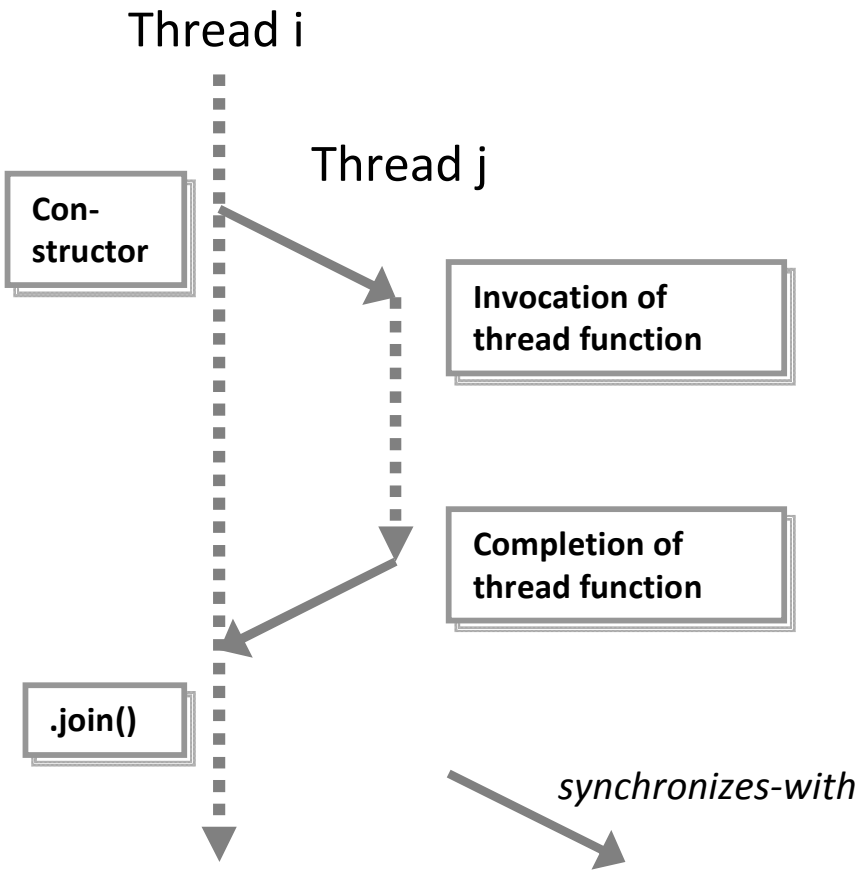
// Thread 1
while(!flag.load(std::memory_order_relaxed))
{ /* spin */ }
atomic_thread_fence(
        std::memory_order_acquire);
cout << val.load(std::memory_order_relaxed);
```



Synchronizes-with and Higher-Level Constructs (1)

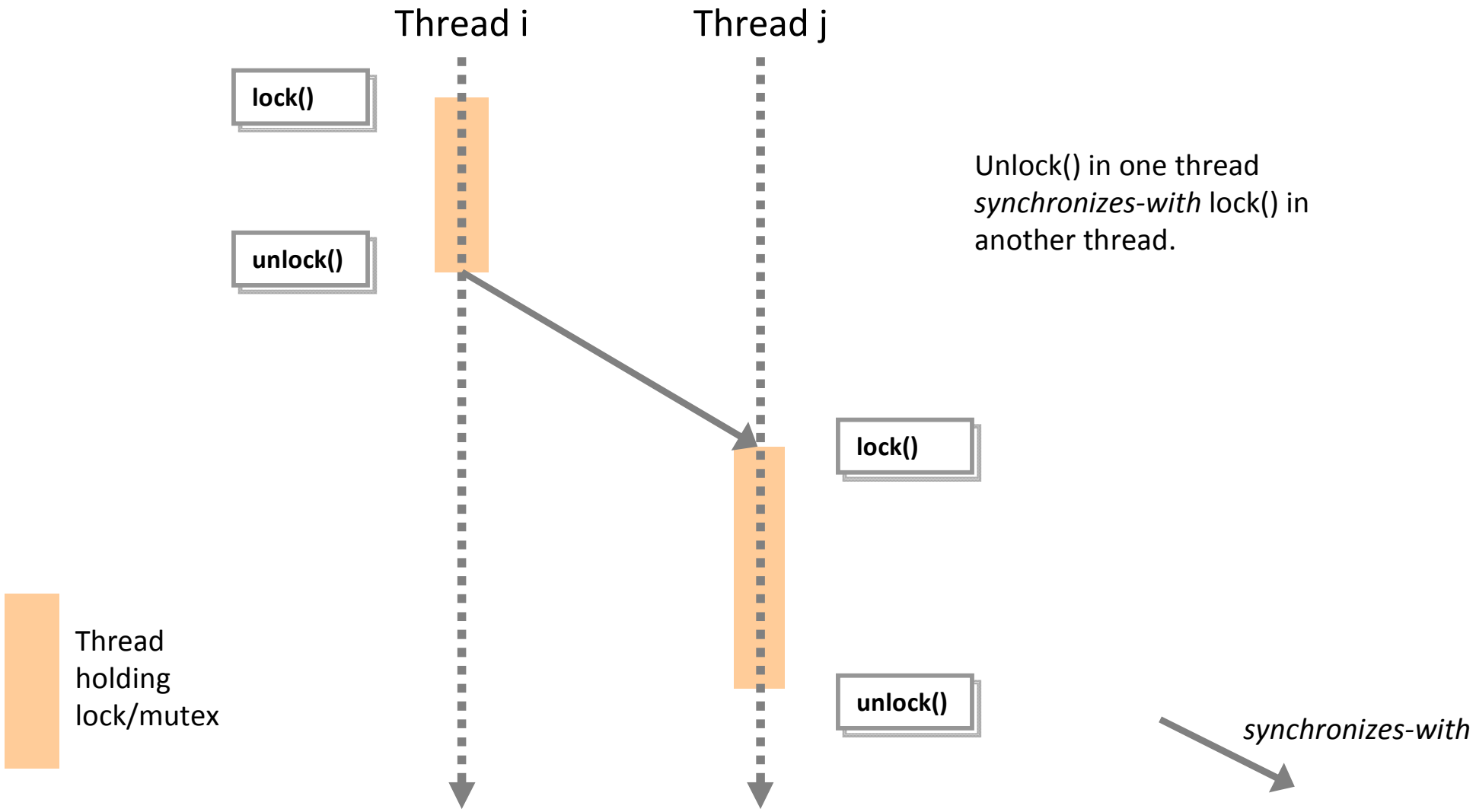
- So far we have discussed how we can achieve the *synchronizes-with* relationship by using low-level mechanism
  - Higher-level constructs also provide *synchronizes-with* as part of their functionality

- Example:
  - Thread creation

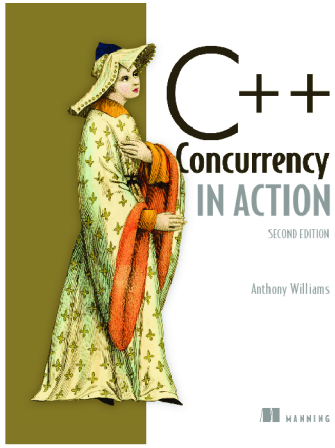


Synchronizes-with and Higher-Level Constructs (2)

- Example: Locks, Mutexes
  - std::mutex realizes a basic lock, with lock() and unlock() functions



Further Reading



- Anthony Williams *"C++ Concurrency in Action"*, 2nd Edition (2017)
  - Chapter 5: “The C++ memory model and operations on atomic types”
  - See download section for a copy of Chapter 5 as a **reading suggestion** and reference for future lectures/labs.
- “Preshing on Programming”, https://preshing.com/
  - Nice blog on issues related to atomic operations, memory consistency, etc.
  - **Reading assignment** as part of this lecture (also see download section): https://preshing.com/20130823/the-synchronizes-with-relation/





- Mara Bos *"Rust Atomics and Locks"* O'Reilly 2023
  - Chapter 3 (Memory Ordering)

Summary

- We discussed the C++ memory model in some detail
  - Was introduced with C++11, along with support for threads and concurrency
  - Refined and updated since
  - The Rust memory model is similar to the C++ one

- C++ atomic operations are at the heart of the memory model
  - 2 Roles:
  - 1) Used to atomically manipulate memory, especially in the form of RMW operations, e.g., compare_exchange aka. compare-and-swap
  - 2) Basis for expressing memory ordering guarantees, establishing the synchronizes-with relationship